

Project 1 Design Document

CS-4240 – Compilers & Interpreters

David Folger, Devin Fromond, Rupesh Seetharaman

9 September, 2025

1. High Level Architecture of the Optimizer
  1. **PARSE:** Parse Tiger-IR text to capture opcode, defined vars, used vars, and labels/branches. Assign stable instruction/location IDs for later lookups.
    - a. **I:** path to Tiger-IR text file
    - b. **O:** IR program with resolved labels and per-function instruction lists (each instruction has ID, def/use, opcode)
  2. **BASIC BLOCKS:** Convert each function's instruction list into basic blocks and build the CFG with predecessors/successors. Compute per-block kill sets and keep a fast map from instruction ID  $\rightarrow$  (block, index).
    - a. **I:** parsed IR program
    - b. **O:** per-function CFG with basic blocks, instruction IDs, and instruction def/use plus per-block kill sets
  3. **REACHING DEF:** Run forward dataflow to compute IN/OUT sets of reaching definitions for each basic block. A def reaches instruction  $i$  if it is in **IN** of the block **B(i)** and is not killed locally within **B(i)** before  $i$  (omit the local-kill check if using an instruction-level CFG). Provide a lookup: given (instruction  $i$ , operand  $y$ ), return the set of instruction IDs  $j$  whose defs of  $y$  reach  $i$ .
    - a. **I:** CFG with def/use/kill sets
    - b. **O:** per-block (or per-instruction) reaching-def facts and a lookup ( $\langle i, \text{operand} \rangle \rightarrow$  set of reaching-def instruction IDs)
  4. **MARK:** Clear marks. Identify critical instructions (treat all branch instructions as critical). Mark each critical instruction and push to **WorkList**. While **WorkList** is not empty, pop  $i$ ; for each operand of  $i$ , use the reaching-defs lookup to find every reaching def  $j$ . If  $j$  is unmarked, mark it and push  $j$ .
    - a. **I:** IR + reaching-defs lookup (and chosen critical set)
    - b. **O:** set of marked (live) instruction IDs
  5. **SWEEP:** For each instruction, if it is not marked, delete it; preserve labels that are jump targets. Emit the remaining IR in program order.
    - a. **I:** IR with mark bits
  6. **O:** optimized Tiger-IR file (dead instructions removed)

We decided to implement the above structure since it builds on best practices as discussed in lecture. Namely, we wish to utilize DCE with reaching definitions as outlined on the lecture slides. It should be noted that the structures above may be differently named than in the codebase.

## 2. Low-Level design decisions and the rationale

- We decided to use Java as our implementation language. Since Java is an object-oriented programming language, it allowed us to easily abstract components of the optimizer, such as Basic Blocks and Control Flow Graphs. In addition, it enabled us to organize our classes and methods in a clear and modular way. This structure also made testing and debugging simpler, as we could more easily isolate issues. Another benefit of using Java was the availability of the provided boilerplate code, which was already written in Java.

## 3. SWE challenges/issues and resolutions

- Some early challenges we encountered involved understanding the structure of Tiger-IR and becoming familiar with the provided boilerplate code.
- Another major challenge was ensuring that a correct Control Flow Graph (CFG) was generated. We verified our CFG implementation by testing it on our own sample IR programs and checking whether the resulting CFG matched the expected structure.
- We also had to ensure that our dead code elimination algorithm was functioning correctly. Similar to how we tested the CFG, we validated it using a variety of test cases. During this process, we discovered that the algorithm occasionally removed code it was not supposed to. As a result, we were able to refine and adjust our implementation to address the issue.
- We also faced general software engineering issues, such as avoiding merge conflicts when working collaboratively. To address this, we communicated our work schedules clearly so that others could avoid pushing code at the same time

## 4. Known bugs/deficiencies (unable to resolve)

All known bugs were resolved.

## 5. Build and usage instructions

**Using a linux environment:** If repository has been freshly cloned, the following must be entered on the command line in order to make the bash scripts executable:

```
$ chmod +x build.sh run.sh
```

**To build:** Building the optimizer is simple. Execute the build.sh script from the command line with:

```
$ ./build.sh
```

**Usage:** To use the optimizer, a run script has been designed to simplify the process. Execute run.sh from the CLI and provide 2 positional arguments, an input .ir file and an output .ir file path using

```
$ ./run.sh input.ir output.ir
```

Optionally, the user may choose to give no args in which case the script will optimize the public test cases and run the interpreter on them. Any other amount of arguments other than 0 or 2 will cause the script to exit and return.

#### 6. Summary of test results for public test cases

We found that the resulting test dynamic instruction counts of our optimizer matched those of Optimizer B. Our optimizer successfully removed dead code based on the lecture algorithm, including definitions for gen, kill, out, and in. Given the results are equal to that of Optimizer B, we can say that our optimizer's performance is the same as the performance of Optimizer B.

```
shakezula@shake-zula: ~/cs4240_proj1/compilers$ sudo ./run.sh
[sudo] password for shakezula:
No args: optimizing public tests and running interpreter...
Number of non-label instructions executed: 434
Number of non-label instructions executed: 1026
Number of non-label instructions executed: 1658
Number of non-label instructions executed: 2322
Number of non-label instructions executed: 2878
Number of non-label instructions executed: 3882
Number of non-label instructions executed: 4306
Number of non-label instructions executed: 5146
Number of non-label instructions executed: 5826
Number of non-label instructions executed: 6514
Number of non-label instructions executed: 83
Number of non-label instructions executed: 61
Number of non-label instructions executed: 11
Number of non-label instructions executed: 89
Number of non-label instructions executed: 119
Number of non-label instructions executed: 139
Number of non-label instructions executed: 151
Number of non-label instructions executed: 247
Number of non-label instructions executed: 139
Number of non-label instructions executed: 11
Done.
```